Ruby defines three levels of access protection for module and class constants and methods:

- *Public*. Accessible to anyone.
- *Protected*. Can be invoked only by objects of the defining class and its subclasses.
- *Private*. Can be called only in functional form (that is, with an implicit receiver of self). Private methods therefore can be called in the defining class and by that class's descendents and ancestors, but only within the same object. See Specifying Access Control, on page 44 for examples.

These levels are invoked with the methods public, protected, and private, which are defined in class Module. Each access control function can be used in three different ways:

- If used with no arguments, the three functions set the default access control of subsequently defined methods.
- With arguments, the functions set the access control of the named methods and constants.
- Since def returns the method name as a symbol, the functions can be used to decorate a method declaration, as in private def foo = 1 + 1.

Access control is enforced when a method is invoked.

Blocks, Closures, and Proc Objects

A code block is a set of Ruby statements and expressions between braces or a do/end pair. The block may start with an argument list between vertical bars. A code block may appear only immediately after a method invocation. The start of the block (the brace or the do) must be on the same logical source line as the end of the invocation.

```
invocation do | a1, a2, ... |
end

invocation { | a1, a2, ... |
}
```

Braces have a high precedence; do has a low precedence. If the method invocation has parameters that are not enclosed in parentheses, the brace form of a block will bind to the last parameter, not to the overall invocation. The do form will bind to the invocation.

Within the body of the invoked method, the code block may be called using the yield keyword. Parameters passed to yield will be assigned to arguments in the block. A warning will be generated if yield passes multiple parameters to a block that takes just one. The return value of the yield is the value of the last expression evaluated in the block or the value passed to a next statement executed in the block.

A block is a *closure*; it remembers the context in which it was defined, and it uses that context whenever it is called. The context includes the value of self, the constants, the class variables, the local variables, and any captured block.

```
class BlockExample
  CONST = 0
  @@a = 3

def return_closure
a = 1
@a = 2
lambda { [ CONST, a, @a, @@a, yield ] }
end
```